Section 4.3 — Storage Engine Deep Dive & Concurrency Control

Enhanced Study Notes | 5 Files in Series (4.1–4.5)

> ***Study Directive (5 files):*** *This is file 3 of 5 in the Section 4 database series. This section is* ***math-heavy*** *(B-Tree page math, LSM write amplification calculations, MVCC snapshot visibility proofs). Give extra weight to* ***worked numeric examples*** *and* ***proof-by-contradiction style explanations*** *for correctness arguments (e.g., why Repeatable Read cannot prevent write skew, proven with a concrete counterexample).*

Executive Overview

Storage engines and concurrency control are where database theory meets mechanical reality. The choice between B-Tree and LSM-Tree determines whether your database is read-optimized or write-optimized. The choice of isolation level determines what anomalies you accept. Both decisions have measurable, quantifiable consequences.

---

1. B-Trees vs LSM-Trees — Quantified Trade-offs

1.1 B+ Tree Structure and Math

```
B+ Tree with order d (max 2d keys per node):
  Typical: page size = 4KB, key+pointer = 16 bytes → ~250 keys per node

  Tree height for N records:
    Height = ⌈log₂₅₀(N)⌉

  Numeric examples:
    N = 1 million   records → height = ⌈log₂₅₀(1,000,000)⌉ = ⌈2.87⌉ = 3 levels
    N = 1 billion   records → height = ⌈log₂₅₀(1,000,000,000)⌉ = ⌈4.31⌉ = 5 levels
    N = 1 trillion  records → height = ⌈log₂₅₀(1,000,000,000,000)⌉ = ⌈5.74⌉ = 6
levels

  Meaning: Even 1 trillion records require only 6 disk I/Os for a point lookup!
```

```
  (Hot internal nodes cached in buffer pool → effectively 1–2 I/Os in practice)

B+ Tree node structure:
                    [30 | 60 | 90]          ← internal node (keys only)
          ↙     ↓     ↓     ↘
              [10|20] [40|50] [70|80] [95|100]  ← leaf nodes (key+data pointer)
                  ↕         ↕          ↕
            (leaf nodes linked as doubly-linked list for range scans)

Range scan efficiency:
  SELECT * FROM orders WHERE order_id BETWEEN 40 AND 80;
  1. Descend tree to leaf containing 40: 3 I/Os
  2. Follow leaf → leaf → leaf pointers until past 80: sequential reads
  Total: 3 random I/Os + sequential scan of matching leaves = very fast

Write cost — page splits:
  Insert value 45 into full leaf [40|50|60|70|80]:
    Leaf split: [40|45|50] and [60|70|80] → 2 leaf writes
    Parent update: add separator key 60 → parent write
    If parent also full: cascades upward (rare, O(1) amortized)
  Write amplification: ~1.5–2x (write 2 pages, logical 1 write)
```

1.2 LSM-Tree Structure and Write Amplification Math

```
LSM-Tree (Log-Structured Merge Tree) — optimized for write throughput:

Architecture:
  ┌──────────────────────────────────────────────────────────┐
  │  MemTable (in-memory, ~64MB)                             │
  │  Sorted red-black tree or skip list                      │
  │  All writes go here first → O(log N) insert              │
  │  WAL written simultaneously for durability               │
  └──────────────────────────┬───────────────────────────────┘
                             │ Flush when full
                             ▼
  ┌──────────────────────────────────────────────────────────┐
  │  L0: SSTables (Sorted String Tables)                     │
  │  Each SSTable is immutable, sorted, with bloom filter    │
  │  L0 → L1 compaction triggered when L0 has 4 SSTables     │
  └──────────────────────────┬───────────────────────────────┘
                             │ Compaction (merge-sort, remove tombstones)
                             ▼
  ┌──────────────────────────────────────────────────────────┐
  │  L1: 10x larger than L0, non-overlapping key ranges      │
  │  L2: 10x larger than L1                                  │
  │  L3–L6: each 10x the previous level                      │
  └──────────────────────────────────────────────────────────┘

Write amplification calculation:
  Level ratio: 10 (each level is 10x larger)
  Write amplification per level: ~10 (each byte written ~10x per level)
  With 6 levels: WA = 10 × 6 = 60x in the worst case
  Meaning: 1 user write → 60 actual disk writes
```

```
  RocksDB tuned: WA ≈ 10-30x in practice

Read amplification (the read penalty):
  Point lookup without bloom filter:
    Must check every SSTable at every level: O(L × files_per_level)
    6 levels × 10 files = 60 file checks per lookup

  With Bloom filter (probabilistic membership test):
    Bloom filter: "Is key X in this SSTable?" → 99% accurate with ~10 bits/key
    Eliminates ~99% of unnecessary SSTable reads
    Expected I/Os per lookup: ≈ 1.06 (usually found in first matching SSTable)
    False positive rate: p = e^(-m×k/n) where m=bits, k=hash_functions, n=elements
    Example: 10 bits/key, 7 hash functions → FP rate ≈ 0.8%

Space amplification:
  Deleted keys leave tombstones until compaction removes them.
  Until compaction: key exists in multiple levels simultaneously.
  Space amplification ≈ 1.1-1.5x (10-50% extra storage)
  B-Tree comparison: ~1.1-1.3x (page fragmentation only)
```

1.3 B-Tree vs LSM-Tree Decision Matrix

Metric	B+ Tree	LSM-Tree
Write throughput	~50K/s	~500K/s (10x)
Write amplification	2–5x	10–30x
Read throughput (point)	~100K/s	~80K/s (with bloom)
Read throughput (range)	Excellent	Good (if compacted)
Space amplification	1.1–1.3x	1.1–1.5x
Write latency p99	Predictable	Occasional compaction spike
Used by	PostgreSQL, MySQL InnoDB, SQLite	RocksDB, Cassandra, InfluxDB, LevelDB

```
Decision guide:

  Choose B-Tree (PostgreSQL/MySQL) when:
    - Read:Write ratio > 80:20
    - Low-latency predictable reads required
    - Range scans are common
    - Update-in-place workloads (update same key repeatedly)

  Choose LSM-Tree (RocksDB/Cassandra) when:
    - Write:Read ratio > 50:50
    - Append-heavy (time-series, events, logs)
    - Write throughput > 100K/s required
```

```
    - Can tolerate occasional compaction pauses
    - Deletions infrequent (tombstones accumulate)

  Real-world validation:
    Facebook MyRocks (RocksDB for MySQL): 50% storage reduction, 10x write
throughput
    vs InnoDB for write-heavy social graph workload.
    Cassandra (LSM): 1M+ writes/sec per node for IoT time-series.
    PostgreSQL (B-Tree): 500K reads/sec for complex OLTP queries.
```

---

2. Transaction Isolation — Anomalies and Levels

2.1 Isolation Anomalies — Concrete Proofs

```
DIRTY READ — Reading uncommitted data (only possible at READ UNCOMMITTED):

  Proof that dirty reads cause incorrect results:
    T1: BEGIN; UPDATE balance SET amount = 0 WHERE account = 'alice';
    T2: SELECT amount FROM balance WHERE account = 'alice'; → returns 0
    T1: ROLLBACK;  ← T1 was a mistake, rolled back
    T2's read of 0 was a lie. Alice still has her original balance.
    Decision made by T2 based on 0 is incorrect.

  Isolation fix: READ COMMITTED
  Mechanism: T2 can only see data committed before its statement began.

NON-REPEATABLE READ — Same query returns different values within one transaction:

  Proof that this violates transaction semantics:
    T1: BEGIN;
    T1: SELECT amount FROM balance WHERE account = 'alice'; → returns 1000
    T2: UPDATE balance SET amount = 500 WHERE account = 'alice'; COMMIT;
    T1: SELECT amount FROM balance WHERE account = 'alice'; → returns 500
    T1: Computes refund = original_balance - current_balance = 1000 - 500 = 500
        But T1 is using TWO different snapshots within one transaction.
        T1's logic assumes it operates on a consistent view of the world. It
doesn't.

  Isolation fix: REPEATABLE READ
  Mechanism: T1's snapshot is frozen at transaction start. T2's commit invisible
to T1.

PHANTOM READ — New rows appear in a repeated query:

  Proof that phantoms can violate invariants:
    T1: BEGIN;
    T1: SELECT COUNT(*) FROM doctors WHERE on_call = true; → returns 2
    T2: DELETE FROM doctors WHERE id = 99 AND on_call = true; COMMIT;
    T2: DELETE FROM doctors WHERE id = 88 AND on_call = true; COMMIT;
```

```
    T1: SELECT COUNT(*) FROM doctors WHERE on_call = true; → returns 0
    T1: Assumes count was stable since it started. It wasn't.

  Isolation fix: SERIALIZABLE
  Or: Predicate locks (lock the range "on_call = true", block inserts/deletes)

WRITE SKEW — The anomaly Repeatable Read cannot prevent (critical!):

  Setup: Rule — at least 1 doctor must be on-call at all times.
         Currently: Doctor Alice (on_call=true), Doctor Bob (on_call=true)

  T_alice: BEGIN (RR);
  T_bob:   BEGIN (RR);

  T_alice: SELECT COUNT(*) FROM doctors WHERE on_call = true; → 2 (sees Alice+Bob)
  T_bob:   SELECT COUNT(*) FROM doctors WHERE on_call = true; → 2 (sees Alice+Bob)

  T_alice: "2 on-call doctors, safe to remove myself"
           UPDATE doctors SET on_call = false WHERE id = 'alice'; COMMIT;

  T_bob:   "2 on-call doctors (from my snapshot), safe to remove myself"
           UPDATE doctors SET on_call = false WHERE id = 'bob'; COMMIT;

  RESULT: 0 doctors on call. Constraint violated.

  WHY REPEATABLE READ FAILS:
    T_alice's snapshot (taken at T_alice BEGIN) shows Bob as on-call = true.
    T_alice commits and sets Alice to false.
    T_bob's snapshot (taken at T_bob BEGIN) shows Alice as on-call = true.
    T_bob commits and sets Bob to false.
    Both transactions saw a consistent snapshot. Both were individually correct.
    Their COMBINATION violates the invariant.
    This is write skew: neither transaction writes the same row as the other,
    but their combined effect violates a multi-row invariant.

  Proof by contradiction that RR cannot prevent write skew:
    Assume RR prevents write skew.
    In RR, T_bob cannot see T_alice's committed changes (that's the whole point of
RR).
    Therefore T_bob cannot detect that T_alice set on_call=false.
    Therefore T_bob proceeds based on stale information.
    Therefore write skew occurs. Contradiction with our assumption. QED.

  FIXES:
    Fix 1: SERIALIZABLE isolation (detects the read-write conflict, aborts one
transaction)
    Fix 2: SELECT ... FOR UPDATE (acquires exclusive lock on the read set)
            T_alice: SELECT COUNT(*) FROM doctors WHERE on_call = true FOR UPDATE;
            Now T_bob's matching SELECT FOR UPDATE blocks until T_alice commits.
            T_bob then sees on_call=false for Alice → count=1 → cannot set itself
false.
    Fix 3: Application-level: re-check the invariant inside the transaction after
update.
```

2.2 Isolation Level Matrix

Level	Dirty Read	Non-Repeatable	Phantom	Write Skew
READ UNCOMMITTED	✅ possible	✅ possible	✅ possible	✅ possible
READ COMMITTED	❌ prevented	✅ possible	✅ possible	✅ possible
REPEATABLE READ	❌	❌ prevented	MySQL: ✅ / PG: ❌	✅ possible
SERIALIZABLE	❌	❌	❌ prevented	❌ prevented

```
PostgreSQL note: PostgreSQL RR prevents phantoms (MVCC snapshot covers the range).
MySQL note: MySQL RR uses gap locks for some cases but not all.
Performance note: SERIALIZABLE aborts ~5–20% of transactions under contention.
  Always implement retry logic when using SERIALIZABLE.
```

---

3. MVCC — Multi-Version Concurrency Control

3.1 PostgreSQL MVCC Internals

```
Every row (tuple) in PostgreSQL has hidden system columns:

  ┌──────────────────────────────────────────────────────┐
  │  t_xmin  | Transaction ID that created this tuple    │
  │  t_xmax  | Transaction ID that deleted/updated it    │
  │          | (0 if still live)                         │
  │  t_ctid  | Physical location (block, offset)         │
  │  data... | Actual column values                      │
  └──────────────────────────────────────────────────────┘

Visibility rule for transaction T reading a tuple:
  Visible if:
    1. t_xmin < T's snapshot XID      (created before my snapshot)
       AND t_xmin is committed        (creator committed)
    2. AND (t_xmax = 0                (not deleted)
            OR t_xmax >= T's XID      (deleted after my snapshot)
            OR t_xmax not committed) (deleter rolled back)

Worked example — MVCC snapshot visibility:

  State: row with (t_xmin=100, t_xmax=0, data="Alice")  ← current row
         row with (t_xmin=50,  t_xmax=100, data="Alice_old") ← old version
```

```
  Transaction T with snapshot_xid = 150 (sees all committed txns < 150):
    Evaluating (t_xmin=100, t_xmax=0):
      t_xmin=100 < 150: YES, and txn 100 committed → condition 1 satisfied
      t_xmax=0: not deleted → condition 2 satisfied
      VISIBLE

  Transaction T' with snapshot_xid = 80 (started before txn 100):
    Evaluating (t_xmin=100, t_xmax=0):
      t_xmin=100 > 80: NO, not satisfied → INVISIBLE
    Evaluating (t_xmin=50, t_xmax=100):
      t_xmin=50 < 80: YES, txn 50 committed
      t_xmax=100 >= 80: YES (deleted after T' started)
      VISIBLE     ← T' sees the old version "Alice_old"

  Result: T sees "Alice". T' sees "Alice_old". Both consistent. No locks required!

MVCC garbage collection (VACUUM):
  Old tuple versions accumulate: (t_xmin=50, t_xmax=100) no longer needed
  when ALL active transactions have XID > 100.
  VACUUM: scans pages, removes dead tuples, marks space as reusable.
  AUTOVACUUM: runs automatically, triggered by dead tuple count.

  Table bloat problem:
    High update rate → many dead tuples → table grows → read performance degrades
    Fix: increase autovacuum aggressiveness, manual VACUUM ANALYZE
```

3.2 MySQL InnoDB MVCC via Undo Log

```
InnoDB approach — undo log chain:

  Current row in clustered index: (id=1, name="Alice_v3", DB_TRX_ID=300)
                                    ↓ (roll pointer)
  Undo log record:                (name="Alice_v2", old_TRX_ID=200)
                                    ↓
  Undo log record:                (name="Alice_v1", old_TRX_ID=100)

  To find version visible to transaction T (snapshot at XID=150):
    Current: TRX_ID=300 > 150 → not visible (created after my snapshot)
    Follow roll pointer → TRX_ID=200 > 150 → not visible
    Follow roll pointer → TRX_ID=100 < 150 → VISIBLE    → return "Alice_v1"

  Undo log purge: background thread removes undo records no longer needed.
  Long-running transactions prevent undo purge → undo log grows → disk fills.
  This is the "long transaction" problem in MySQL.
```

---

4. Optimistic vs Pessimistic Concurrency

4.1 OCC — When Retries Are Acceptable

```
Optimistic Concurrency Control assumes conflicts are RARE.
No locks held during the transaction. Conflict detected at commit time.

Version-based OCC:

  CREATE TABLE accounts (
    id       BIGINT PRIMARY KEY,
    balance  NUMERIC,
    version  INT DEFAULT 0  ← optimistic version counter
  );

  def transfer(from_id, to_id, amount, max_retries=5):
    for attempt in range(max_retries):
      # Read phase (no locks)
      from_acct = SELECT * FROM accounts WHERE id = from_id
      to_acct   = SELECT * FROM accounts WHERE id = to_id

      if from_acct.balance < amount: raise InsufficientFunds()

      # Compute new values
      new_from_balance = from_acct.balance - amount
      new_to_balance   = to_acct.balance + amount

      # Write phase — atomic version check
      rows = UPDATE accounts SET balance=new_from_balance, version=version+1
             WHERE id=from_id AND version=from_acct.version
      rows += UPDATE accounts SET balance=new_to_balance, version=version+1
              WHERE id=to_id AND version=to_acct.version

      if rows == 2: return SUCCESS  ← both updated (no conflict)
      # If rows < 2: someone else updated, retry
      sleep(0.01 * 2**attempt)  ← exponential backoff
    raise MaxRetriesExceeded()

  When OCC wins over locking:
    Low contention (< 5% conflict rate): OCC throughput >> locking
    Read-heavy transactions: no locks on reads = massive concurrency
    Short transactions: retry cost low

  When OCC loses:
    High contention (> 20% conflict rate): too many retries → livelock risk
    Long transactions: read phase takes seconds → conflict probability high
```

4.2 Pessimistic Locking — SELECT FOR UPDATE

```
Pessimistic: assume conflicts ARE likely. Acquire lock before reading.

  BEGIN;
  SELECT * FROM accounts WHERE id = 42 FOR UPDATE;
  -- Lock held: other transactions block here until this COMMIT/ROLLBACK
  UPDATE accounts SET balance = balance - 100 WHERE id = 42;
  COMMIT;  -- lock released

Lock modes in PostgreSQL:
  FOR UPDATE:         exclusive, blocks all other FOR UPDATE, FOR SHARE
  FOR NO KEY UPDATE:  exclusive on row, but allows FOR SHARE
  FOR SHARE:          shared, allows multiple readers, blocks FOR UPDATE
  FOR KEY SHARE:      weakest, allows most concurrent access

Deadlock example and detection:
  T1: LOCK accounts WHERE id = 1 (holds lock A)
  T2: LOCK accounts WHERE id = 2 (holds lock B)
  T1: LOCK accounts WHERE id = 2 → WAIT (waiting for B)
  T2: LOCK accounts WHERE id = 1 → WAIT (waiting for A)
  Circular dependency → DEADLOCK

  Detection: PostgreSQL detects cycles in wait-for graph (checked every
deadlock_timeout=1s)
  Resolution: Abort the transaction with smaller XID (newer transaction)
  Prevention: Always acquire locks in same order (e.g., lowest ID first)
    T1: LOCK id=1 then id=2
    T2: LOCK id=1 (blocks until T1 done) then id=2
    No deadlock possible.

Throughput comparison at different contention levels:
  Conflict rate | OCC throughput | Pessimistic throughput
  0%            | 100%           | 70% (lock overhead)
  5%            | 95%            | 70%
  20%           | 80% (retries)  | 65%
  50%           | 50% (livelock risk) | 60%
  Rule of thumb: OCC wins when conflict rate < 10–15%.
```

---

5. Distributed Locking — When You Need It and When You Don't

5.1 Redlock — Math and Failure Analysis

```
Redlock algorithm (Martin Kleppmann designed, Redis community uses):

Setup: 5 independent Redis nodes (N=5, quorum = N/2+1 = 3)

Acquire:
```

```
  1. Note time T_start = now()
  2. SET lock:order-123 <unique_token> NX PX 30000  to all 5 nodes (parallel)
  3. Count successful SETs: call this k
  4. T_elapsed = now() - T_start
  5. If k >= 3 AND T_elapsed < 30000ms:
       lock_validity = 30000 - T_elapsed - clock_drift_buffer (≈ 300ms)
       LOCK ACQUIRED with TTL = lock_validity
  6. Else: release on all nodes (send DEL with unique_token), FAILED

Release:
  Lua script on each node (atomic):
    if GET lock:order-123 == <unique_token>: DEL lock:order-123
  (Check token prevents releasing someone else's lock)

Failure analysis — GC pause scenario:
  Client A acquires lock, lock_validity = 29s
  Client A's JVM GC pause lasts 32 seconds (longer than TTL!)
  Redis auto-expires the lock after 30s
  Client B acquires same lock (legitimate, A's lock expired)
  Client A resumes from GC pause, believes it still holds lock → WRONG
  Client A and B both believe they hold the lock → unsafe!

  Mitigation: Fencing tokens (monotonic counter)
    Each lock acquisition returns an incrementing token.
    Storage/resource uses the token: only accepts operations with token >=
last_seen.
    Client A's stale token (e.g., 50) < Client B's token (51) → rejected.
    Even if Client A believes it has the lock, writes are rejected.
```

5.2 Distributed Lock Comparison

Mechanism	Correctness	Performance	Complexity	Best For
Single Redis	Low (SPOF)	Very fast	Low	Non-critical, dev env
Redlock (5 nodes)	Medium (GC issue)	Fast	Medium	General distributed lock
Redlock + fencing	High	Fast	High	Safety-critical
ZooKeeper/etcd	High (Raft)	Medium	High	Leader election, config
DB row lock	High	Slow	Low	Already using a DB

6. Interview Use Cases & Design Problems

Problem 1 — Why Does Your B-Tree Index Hurt Write Performance?

Problem statement: A team added 15 indexes to a high-write `events` table to support various queries. Write throughput dropped from 80K/s to 8K/s. Why, and how do you fix it?

Solution:

```
Root cause — every write maintains every index:
  INSERT INTO events (user_id, type, ts, payload) VALUES (...)
  → Write 1 record to heap (data table)
  → Write to index on user_id
  → Write to index on type
  → Write to index on ts
  → Write to index on (user_id, type)
  → Write to index on (user_id, ts)
  ... × 15 indexes = 16 write operations per INSERT

  At 4KB pages and 15 indexes:
    1 logical insert → 16 page writes × 8KB average = 128KB of I/O per row
    80K rows/s × 128KB = 10GB/s disk I/O → far exceeds disk bandwidth

Diagnosis:
  SELECT schemaname, tablename, indexname,
pg_size_pretty(pg_relation_size(indexrelid))
  FROM pg_stat_user_indexes
  WHERE tablename = 'events'
  ORDER BY pg_relation_size(indexrelid) DESC;

  SELECT indexrelname, idx_scan, idx_tup_read, idx_tup_fetch
  FROM pg_stat_user_indexes
  WHERE tablename = 'events'
  ORDER BY idx_scan ASC;  ← indexes with 0 scans = unused

Fix:
  Step 1: Identify and DROP unused indexes (0 scans since last stats reset).
          Keep only indexes used by actual queries.

  Step 2: Consolidate redundant indexes:
    (user_id), (user_id, type), (user_id, ts) → replace with (user_id, ts, type)
    (multicolumn satisfies prefix queries too: WHERE user_id = X is served by
(user_id, ts, type))

  Step 3: Partial indexes for write-hot tables:
    Instead of index on all events, index only recent events:
    CREATE INDEX idx_events_recent ON events (user_id, ts)
    WHERE ts > NOW() - INTERVAL '7 days';
    ← Maintained only for ~7-day window, not all history

  Step 4: Consider LSM-Tree engine (RocksDB / MyRocks) for pure write-heavy
tables.
```

```
    B-Tree → LSM-Tree: write throughput 8x improvement for append-only workloads.

Result: 15 indexes → 5 indexes → write throughput recovers to ~60K/s.
```

Problem 2 — Prove SERIALIZABLE Prevents the Doctor Write Skew

Problem statement: Your hospital system uses REPEATABLE READ. A bug allows 0 doctors on call (write skew). Show with SQL that SERIALIZABLE prevents this, and explain the mechanism.

Solution:

```
Setup:
  CREATE TABLE doctors (id TEXT PRIMARY KEY, on_call BOOLEAN);
  INSERT INTO doctors VALUES ('alice', true), ('bob', true);

With REPEATABLE READ — write skew occurs:
  Session A:                                Session B:
  BEGIN ISOLATION LEVEL REPEATABLE READ;  BEGIN ISOLATION LEVEL REPEATABLE READ;
  SELECT COUNT(*) FROM doctors
  WHERE on_call = true;  → 2
                                            SELECT COUNT(*) FROM doctors
                                            WHERE on_call = true;  → 2
  UPDATE doctors SET on_call = false
  WHERE id = 'alice'; COMMIT;
  ← Alice now off-call. 1 doctor left.
                                            UPDATE doctors SET on_call = false
                                            WHERE id = 'bob'; COMMIT;
                                            ← Bob off-call. 0 doctors!
  Final state: 0 on-call doctors. Invariant violated.

With SERIALIZABLE — write skew prevented (PostgreSQL SSI):
  Session A:                                Session B:
  BEGIN ISOLATION LEVEL SERIALIZABLE;     BEGIN ISOLATION LEVEL SERIALIZABLE;
  SELECT COUNT(*) FROM doctors
  WHERE on_call = true;  → 2
  ← PG registers: A read predicate "on_call = true"

                                            SELECT COUNT(*) FROM doctors
                                            WHERE on_call = true;  → 2
                                            ← PG registers: B read predicate
"on_call = true"

  UPDATE doctors SET on_call = false WHERE id = 'alice'; COMMIT;

                                            UPDATE doctors SET on_call = false
                                            WHERE id = 'bob';
                                            COMMIT;
                                            ERROR: could not serialize access due to
                                            read/write dependencies among
transactions
```

```
                                          ← PostgreSQL SSI detected: B's read of
                                            on_call=true was invalidated by A's
commit.
                                            B must retry.

Mechanism — Serializable Snapshot Isolation (SSI):
  PostgreSQL tracks read predicates (what ranges each transaction read).
  When a transaction commits, PostgreSQL checks:
    "Does any other transaction have a read predicate that overlaps with what I
wrote?"
  If yes: one of the transactions must abort (the later committer loses).
  This detects write skew because: A wrote to rows B read, and B wrote to rows A
read.
  The cycle (A → B → A) indicates serializability violation.

When to use SERIALIZABLE:
  ✅ Financial systems with multi-row invariants
  ✅ Reservation systems (overbooking prevention)
  ✅ Any invariant that spans multiple rows
  ❌ High-throughput OLTP (5–20% abort overhead, always add retry)
```

Problem 3 — Choosing Between OCC and Locking for a Flash Sale System

Problem statement: Design the concurrency control for a flash sale: 100 units of a product, 50,000 simultaneous buyers. Every purchase must decrement inventory. How do you prevent overselling and maximize throughput?

Solution:

```
Analysis:
  50,000 buyers competing for 100 units = extremely high contention.
  After 100 sales, all remaining transactions should fast-fail.
  This is worst case for OCC (nearly 100% conflict rate after inventory depletes).

Approach A — Pure OCC (WRONG for this case):
  All 50,000 read inventory = 100.
  All 50,000 try to UPDATE WHERE qty = 100.
  1 succeeds. 49,999 retry.
  Each retry reads new value (99), 49,998 try again.
  ...100 rounds of retries for 100 units.
  Result: 100 successful purchases after ~100×49,999 failed attempts =
catastrophic.

Approach B — Pessimistic locking (better but slow):
  SELECT qty FROM inventory WHERE product_id = 'X' FOR UPDATE;
  → serializes all 50,000 requests. Throughput: 1 purchase/lock_cycle.
  At 5ms/cycle: 100 units × 5ms = 500ms total to sell out. 49,900 wait.
  Acceptable but queue depth is enormous.

Approach C — Redis atomic decrement (BEST for this pattern):
```

```
  Pre-load inventory count into Redis:
    SET inventory:product_X 100

  Purchase attempt (Lua script — atomic):
    local qty = redis.call('GET', 'inventory:product_X')
    if tonumber(qty) <= 0 then return -1 end  -- sold out
    return redis.call('DECR', 'inventory:product_X')

  Redis: single-threaded command execution = no race condition.
  Throughput: ~500K operations/second on Redis.
  All 50,000 buyers: 100 get qty > 0 (success), 49,900 get -1 (sold out).
  Total time to sell out: 100 / 500,000 = 0.2ms.

  DB write (async, after Redis decrement succeeds):
    On Redis decrement success → enqueue DB write to order queue.
    Kafka consumer: write order to PostgreSQL durably.
    If DB write fails: compensate (re-increment Redis, notify user).

  Trade-off: Redis is in-memory. Crash before DB write = inventory inconsistency.
  Mitigation: Redis persistence (AOF) + idempotent order writes.

Summary — when to use each:
  OCC:             Low contention (< 10% conflict), long read phases
  Pessimistic lock: Medium contention, short critical sections
  Redis atomic:    Extreme contention, counter-style operations, performance-
critical
  DB SERIALIZABLE: Correctness-critical multi-row invariants, tolerate retries
```